Flight Dynamics Onboarding

Fall 2026

Getting Started

Introduction

Welcome to Flight Dynamics! We are so glad to have you aboard. Just to help expedite a few processes, we have prepared this short onboarding project to help you get acquainted with some of what you are going to be doing. While we have provided you with system parameters and instructions, some parts of this project will require a bit of research to figure out. We have linked some potentially useful sources throughout the document, but if you have any lingering questions you just can not seem to figure out, please reach out to **Gary Huang** and **Pradyunn Kale** on Slack. We will be happy to help you out!

Your goal will be to make two simple 3 degree-of-freedom (DoF) rocket model in MATLAB. The first 3-DoF will essentially be a high-school level projectile problem, but with additional aerodynamic forces added in to make it a bit more challenging. The second will be a more complex project, where you will model the rotational dynamics of a tennis racket. Upon successful completion, you will have some of the basic knowledge and training that will be needed to begin your journey with the Flight Dynamics subteam.

Objectives

To help further motivate you and improve clarity, we are including the learning objectives and knowledge you will have gained upon completion of this onboarding project.

- Gain a familiarity with MATLAB and its built-in functions
- Understand the core concepts of a simulation model such as a 3-DoF, and how to construct them

- Utilize git / GitHub to safely upload, save, and share a project
- Effectively document processes and rationalize choices both in-code and within a technical write up
- Learn to efficiently research unfamiliar topics using available internal and external resources

Deliverables

As you conclude your work, you should have a collection of documents, graphs, and code you put together along the way. We have listed them all here, so you know what you should have at the end of it all. Every deliverable here will be discussed in more detail in the following sections, so please refer to those for clarification!

- Your 3-DoF code with the following outputs:
  - Graph of the rocket trajectory
  - Graph of the rocket velocity
  - Apogee (in meters)
  - Animation of the rocket
  - Anything else you would like to highlight
- Your Rotational 3-DoF should have the following:
  - An animation of the tennis racket over time
  - A graph showing the total energy of the system
- A `README.md` file, explaining what you did and why you did it in order to provide documentation for your code. If you've never written a readme or used markdown, you can read more here.
- All of these should consolidated in a GitHub repository!

Again, you will be inviting all leads to your GitHub project repository. When your code is ready for review, please alert both **Gary Huang** and **Pradyunn Kale** through a Slack message! We will both review your project, and then sit down with you to chat about your design. This way, we can directly converse with you to not only ask questions and hear your thought process, but also start to get to know you!

Your First Mission: The 3-DoF

A 3-DoF model allows us to understand the movements of a rocket in the xy-plane. A 3-DoF simulation is a good starting point for understanding the stability of a rocket and predicting its apogee. The 3-DoF, as the name implies, has 3 directions in which it is free to move:

1. left-right (x-axis)
2. up-down (y-axis)
3. rotation (about z-axis)

Luckily, the 3-DoF is quite simple. You have covered all of the relevant physics in your high school courses, or are doing so now in your Physics 172 class! There are only a few considerations to implement these equations into a 3-DoF simulation.

Forces and Moments

We will start with a free body diagram of the 3-DoF. A free body diagram should always be your first step in a dynamics problem. We are giving you the diagram for the rocket in Figure 1, but in the future you may have to come up with this yourself!

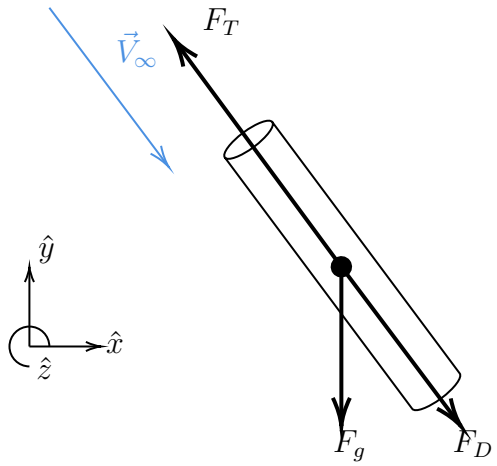

Figure 1: The 3-DoF Forces

Some of these symbols may be familiar to you, but just to get us all on the same page:

F_T	Thrust Force
F_g	Gravitational Force
F_D	Drag Force
V_∞	Freestream Velocity

For our example here, all of the vectors (aside from the moment) will lie in the xy-plane. For the aerodynamic forces, you will have something of the following form:

$$F_D = \frac{1}{2}\rho {V_\infty}^2 S C_D$$

For simplicity, there will be no wind. As such, the lift will remain zero throughout the flight, as there will never be a perturbation to induce an angle of attack. I will leave the rest of the forces to your discretion. You should use your own judgment to make good assumptions about these (and document these assumptions in your `README.md` file).

Rocket Specifications

Luckily for you, you will not have to design a rocket completely from scratch! Almost all the parameters of the vehicle necessary to construct your simulation will

be provided in the Table below. Should you feel a parameter required by your code is missing, please reach out to us and ask.

Rocket Parameters	
Thrust	2000 N
Mass	40 kg
Burn Time	5 s
Outer Diameter	0.1 m
Drag Coefficient	0.75
Initial Velocity	5 m/s

Simulation

So now you have all of the parameters of the rocket and the equations that describe the forces on the rocket. The final step is to take all of these and put them into a simulation so that the trajectory of the rocket can be understood.

In a complex scenario like this, an analytical solution is impossible[1]. In lieu of an analytical solution, we will use an approximation known as *numerical integration*. We will not dive deeply into numerical integration here, as we have done so extensively in Volume I. The main idea is that you must express your forces using first-order differential equations. These first-order differential equations can be derived from your governing equations of motion. Again, refer to Volume I for the derivation of these equations of motion.

Further Improvements

This onboarding project has made quite a few simplifications. Should you wish to dive further into a specific topic, or show off your coding prowess and expertise as a flight dynamicist, you are more than welcome to make additional improvements to the base project! Just please be sure to document everything! A few basic ideas are provided below, to help you brainstorm!

[1] See volume I for more details

- Include a study of the effects changing propellant mass has on the system
- Include an apogee comparison for a range of launch angles to highlight impact on performance
- Use RasAero to generate a look up table for aerodynamics (this could introduce Mach dependence)
- And more!

Remember the learning objectives outlined in the Getting Started section: this project is meant for you to gain a familiarity with MATLAB, GitHub, and the technical skills you will use as a member of Flight Dynamics. So do not be afraid to experiment, practice, and play around a little!

Your Second Mission: Tennis Racket

The second project that you will embark on is the study the rotational dynamics of a tennis racket. Before we start, let's define our axes conventions for the tennis racket, which are shown in Figure 2:

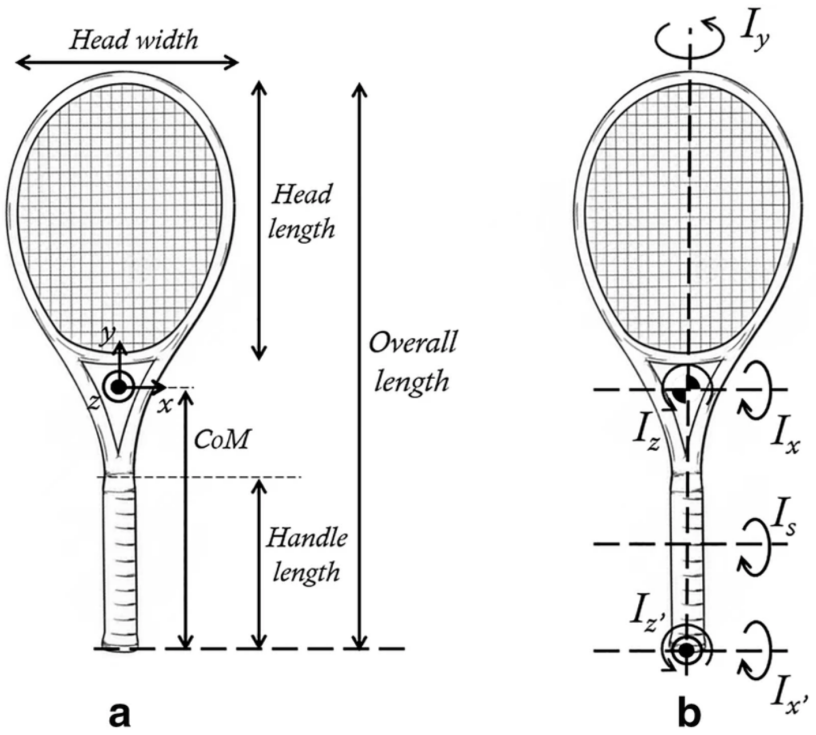

Figure 2: Tennis Racket

As you might know from experience in real life, a tennis racket displays an interesting phenonomon – if you were to throw it rotating along the x-axis shown above, it does an interesting spiral through the air! It does not do this if you rotate about the y or z-axes. That effect is known as the **Dzhanibekov** effect. We are going to demonstrate the phenomenon through simulation.

For simplicity, we will ignore the position and velocity of the racket (although you

may add them if you like!). We will only focus on the rotational dynamics of the object. Our *state vector*[2] needs to define two things: the initial rotation, as well as the angular velocity of the object. We will numerically integrate this as we had done for the simple 3-DoF rocket. The angular velocity components are expressed in terms of the tennis racket axes:

$$\vec{\omega}_0 = \begin{bmatrix} \pi \\ 0.05 \\ 0.05 \end{bmatrix} rad/s$$

This represents a primary rotation about the x-axis, with some small rotation in the y and z axes which model the fact that we cannot throw the object perfectly along a single axis. The next question is how to encode the 3D rotation of the tennis racket. This turns out to be quite the challenge. One popular method, which we use in our full 6-DoF simulation (and is also used in many games & simulation environments), is *quaternions*. A quaternion is a 4-dimensional object which can be used to parameterize a rotation. We'll start our simulation with no rotation, which is given by the following quaternion:

$$\vec{q}_0 = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Our full state vector will contain both $\vec{\omega}_0$ and $\vec{q}_0$. We can arrange them in any order we want, as long as we keep track of the components. The combination $\left[\vec{\omega}, \vec{q}\right]$ is referred to as the X:

$$X = \begin{bmatrix} \vec{\omega} \\ \vec{q} \end{bmatrix}$$

The initial value, X_0, written out in full, is:

$$\vec{X}_0 = \begin{bmatrix} \pi \\ 0.05 \\ 0.05 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

[2] A state vector is all the information we need at one time in order to propagate the simulation to a future time. We typically express it as a column vector.

Dynamics

Next, we need to consider the dynamics of the system. We'll pretend that there are no moments acting on the system after we let go of the racket, which makes things easier. If you are interested in seeing a derivation, Volume I or Volume II of the Flight Dynamics Bible provide derivations (these are linked in Additional Resources section). Here, we'll just provide those equations:

$$\dot{\omega}_x = \frac{(I_y - I_z)\,\omega_y\omega_z}{I_x}$$
$$\dot{\omega}_y = \frac{(I_z - I_x)\,\omega_z\omega_x}{I_y}$$
$$\dot{\omega}_z = \frac{(I_x - I_y)\,\omega_y\omega_x}{I_z}$$

For the quaternion:

$$\dot{\mathbf{q}} = \frac{1}{2}\begin{bmatrix} 0 & -\omega_x & -\omega_y & -\omega_z \\ \omega_x & 0 & \omega_z & -\omega_y \\ \omega_y & -\omega_z & 0 & \omega_x \\ \omega_z & \omega_y & -\omega_x & 0 \end{bmatrix}\mathbf{q}$$

For our simulation, we will need the I parameters as well. Those are the *moments of inertia* along each axis, and tell us about the resistance to rotation. You should use the following values:

$$I_x = 0.02 \text{ kg} \cdot \text{m}^2$$
$$I_y = 0.003 \text{ kg} \cdot \text{m}^2$$
$$I_z = 0.027 \text{ kg} \cdot \text{m}^2$$

This code is a bit more complex than the 3-DoF rocket. As a result, we recommend trying `ode45` or something similar to integrate the system. You can find more information about these integration methods online or in the Flight Dynamics Bible. After you get outputs from whichever integrator you choose, run the quaternion sequence through the `rotationViewer` code that we've provided you to see an animation.

Additional Resources

There may be some parts of this project which require you to complete a task you are not familiar with. Should you find yourself in this situation, we want to remind you that there are additional resources within the Flight Dynamics confluence, as well as a plethora of information available online for you to use and reference. For instance, what we affectionately call the Flight Dynamics Bible (written by former team leads Hudson Reynolds & Preston Wright). The first text in this trilogy details all the necessary knowledge to construct a 6-DoF model. As such, much of the information contained there will likely be useful to you here. You can find this text in the Confluence. If you use another source, be sure to give credit where credit is due and reference it within your documentation!

If this is the first time you are working in MATLAB, there is a document that goes over some of the basic MATLAB syntax. Mathworks themselves have an onramp course on MATLAB which gives a more thorough introduction to the language.

Wrapping it All Up

Once you have completed your project, you should do a few things. First, double and triple check to ensure your outputs and results are physically reasonable and you have a sufficient amount of documentation. Once you have done so, if you have not already, add both **Gary Huang** and **Pradyunn Kale** to your GitHub repository (ask if you are not sure how to do so!), and then message both of us on Slack in a group huddle. We will take a gander at your project, and schedule a chat with you to give some constructive feedback and comments on possible improvements, as well as highlight all the things we liked! While you wait for us to review your project, go ahead and check out the 6-DoF itself to gain a familiarity with it. You should also take a glance in Confluence at the available resources and check out the task list to consider what aspect(s) of flight dynamics and/or GNC engineering you find most interesting!

And most importantly - congratulations on finishing your onboarding and welcome again to Flight Dynamics! We can not wait for all the cool things you are going to be a part of.